BrightTime

DOCUMENTATION

Technical Reference

Architecture, data and security, the API, configuration, operations and development

For the people who build and run BrightTime

BrightTime is a countywide platform for out-of-school time. Families and young people find after-school, summer and weekend activities; the organisations that run them take registrations and record attendance; and the county's network organisers vet every organisation and listing.

As built through decision D66 · 22 October 2026

Kettle County, NY (demonstration region)

Contents

Overview 4

- The system in one paragraph 4
- Status by area 5

Architecture 6

- Hosts 6
- Solution layout 7
- Modules 8
- How modules talk 9
- Tenancy 10
- The request pipeline 10
- The outbox and the job queue 11
- The finder 11
- The app 12
- Localisation 12

Data and security 13

- The schema 13
- Database roles 14
- Row-level security 14
- Rules the database enforces 15
- Encryption and keys 16
- Signing in 17
- Permissions 18
- Audit 18
- Youth safety and privacy, end to end 20
- Known gaps 20

API reference 21

- Conventions 21
- Errors 21
- Authentication 22
- Public reads 23
- Your account 24
- Children and consent 25
- Family profile and shares 25
- Applications, waivers and payments 26
- Attendance 26
- Messages and devices 27
- Learning: portfolio, badges and playlists 27
- Learning: goals, skills and projects 28
- Organisation routes 28
- Organiser and webhook routes 30

Configuration 31

Connections 31
Tenancy and addresses 32
Keys 32
Sign-in and sessions 33
The app's links 33
Mail, push, payments and reports 34
The finder 34
The worker 35
Health 35
A production checklist 35

Operations 36

The local stack 36
The database 36
Deploying 37
Health and the worker 38
Keys 39
Payments 39
Push notifications 40
Mail 40
Badges 40
Not yet in place 40

Development 42

Tools 42
Building 42
Tests 43
CI 44
Adding a feature slice 44
Conventions the build enforces 45

CHAPTER 1

Overview

How BrightTime is built and run, as of D66 (2026-10-22). This set describes the system as it is. For why it is that way, follow the decision numbers (D1 to D66) into plan/08-decisions.md. For the plan as approved, see plan/.

Document	Read it to…
Architecture	understand the three hosts, the modules, how modules talk, tenancy and the request pipeline
Data and security	understand the schema, row-level security, encryption and keys, sign-in, permissions, audit and the youth-safety rules the database enforces
API reference	call `/api/v1`: conventions, errors, paging, idempotency, and every route
Configuration	configure a host: every setting, its default, and which hosts read it
Operations	run the stack, apply the schema, seed, keep the worker healthy, rotate keys, set up payments, push and mail
Development	build, test, and keep CI green, including the conventions the tests enforce

The system in one paragraph

BrightTime is a modular monolith on .NET 10 and PostgreSQL 18 with PostGIS. Three hosts share one database: **Web** (Blazor static server rendering: the public finder and the consoles), **Api** (`/api/v1`, used by the MAUI app and integrations) and **Jobs** (the worker: outbox delivery and background jobs). Each module owns one schema and one EF Core context, and talks to other modules only through their `.Contracts` assemblies or through outbox messages. Tenants are told apart by host name; row-level security isolates them in the database, and a second, restrictive policy keeps every youth-record table closed to a request with no signed-in actor. The DDL in `db/schema/` is the source of truth, and EF is mapped to it.

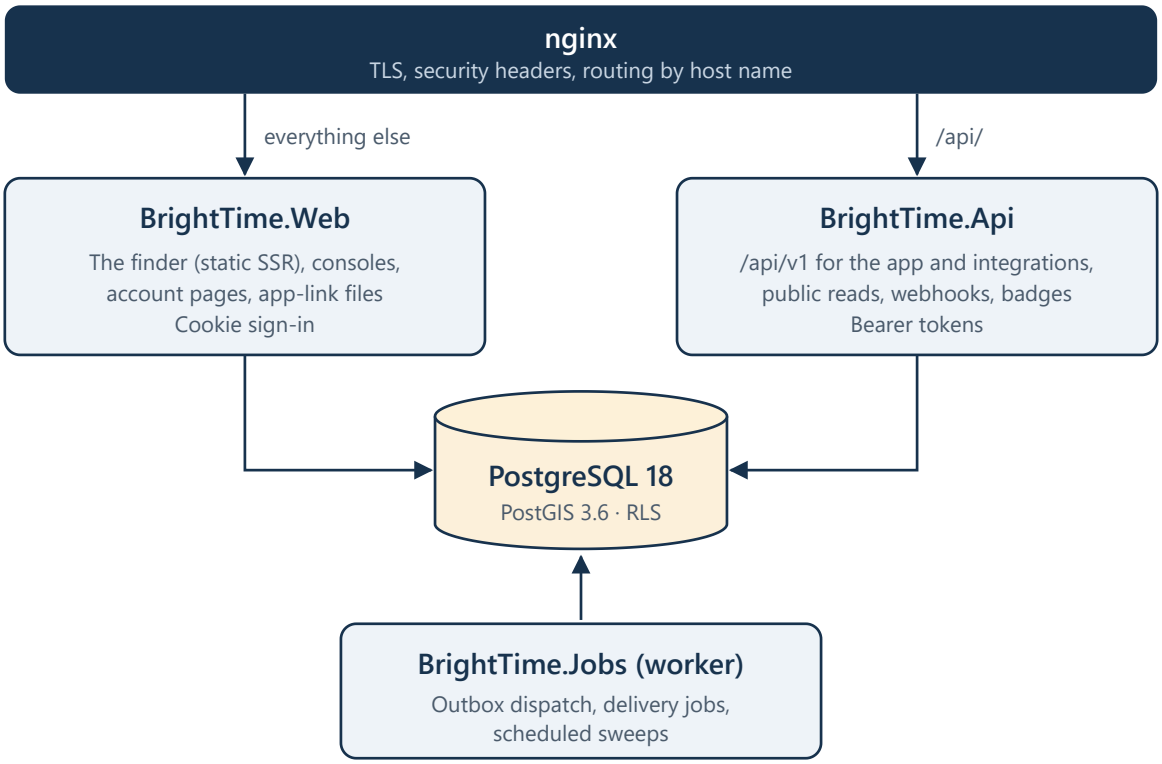
Status by area

Area	Built	Milestone
Tenancy, finder, consoles, sign-in, audit, localisation, import	Yes	M1
Public read API, consent, youth accounts, passkeys, notifications, the app	Yes (app: Android built and run on the emulator; iOS written, not compiled)	M2
Family profile, shares, forms, eligibility, applications, waivers, decisions	Server and provider console; in the app, the household and its shares (D71) and applying, waivers and offers (D72)	M3a
Attendance (with offline sync), messaging, fees and payments (manual and Stripe), reports	Server, provider console and the app's offline roster; in the app, a family's fees and payments (D73, D74); other family screens not yet	M3b
Participation and portfolio (D65), Open Badges (D66), playlists and gated listings (D67), portfolio exports as PDF, OB 3.0 and CLR 2.0 (D68), goals, skills and projects with staff approval (D69)	Server only	M4
Community spaces, workforce, landscape, analytics, integrations	Schema only	M5 to M8

CHAPTER 2

Architecture

Hosts



Host	Project	What it serves	Database logins
Web	`src/BrightTime.Web`	The public finder (works with JavaScript off, D7), listing, collection, neighbourhood and organisation pages, account pages (sign-in, reset, claim, security, terms), the provider and organiser consoles, `/.well-known` app-link files	`Finder:ConnectionString` (a member of `brighttime_app`; enters `brighttime_finder` for search, D13), `Auth:ConnectionString` (`brighttime_auth`)
Api	`src/BrightTime.Api`	`/api/v1`: JSON for the app, public reads, webhooks, Open Badges documents	`ConnectionStrings:App`, `ConnectionStrings:Auth`
Jobs	`src/BrightTime.Jobs`	No HTTP surface. Dispatches the outbox, runs delivery and scheduled jobs. Web's and Api's readiness probes report its queue lag	`ConnectionStrings:Jobs` (`brighttime_jobs`)

The three hosts are composed from the same modules, so a module's rules hold whichever host runs it. `BrightTime.Hosting` holds what Web and Api share: tenancy, the principal, audit, health, localisation middleware, mail and the data-protection key ring.

A one-shot **seed** container (`src/BrightTime.Seed`) loads the Kettle County tenant (seed.md).

Solution layout

Path	Contents
`src/BrightTime.SharedKernel`	Ids, `Result` and `Error`, paging cursors, `IClock`, privacy attributes (`[YouthData]` , `[YouthRecord]`), text helpers
`src/BrightTime.Abstractions`	Interfaces for things outside the process: `IKeyVault` , payments, geocoding, storage, screening, reporting, messaging
`src/BrightTime.Infrastructure`	EF base context and tenancy interceptor, outbox, job runner, audit trail, field encryption, key vault, email
`src/BrightTime.Hosting`	Shared host plumbing (above)
`src/BrightTime.Localisation`	The six catalogues (`Strings/*.json`) and `Text` , with no ASP.NET dependency so the app can use them
`src/BrightTime.Modules.<Name>` and `.Contracts`	One module each (below)
`src/BrightTime.Reporting`	Report rendering (PDF with QuestPDF, Excel, CSV) and the provider report runner
`src/BrightTime.Ui.Shared` , `.Ui.Design` , `.Mobile`	The app: Razor screens and API client, styles, and the MAUI host
`src/BrightTime.Seed`	The Kettle County seed
`db/schema/`	The DDL, applied in file order by `db/apply-schema.sh`
`db/queries/` , `db/explain/`	The finder's reference query, and its measured plans
`deploy/`	Dockerfiles, `compose.yml` , nginx and Postgres init
`tests/`	One project per area (see development.md)

Modules

Each module owns one PostgreSQL schema and one EF Core `DbContext` (`SchemaName`, its own migrations-history table name). Other modules see it only through its `.Contracts` assembly.

Module	Schema	Owns	State
Platform	`platform`	Tenants and domains, geography (neighbourhoods, schools), taxonomy, wording overrides, the outbox and job queue, audit, data-protection keys	Built
Identity	`identity`	Accounts, credentials and passkeys, refresh tokens, roles, guardian links, consent, youth capabilities, terms	Built
Directory	`directory`	Organisations, sites, hours, memberships and invitations, clearances (status only), vetting	Built
Opportunities	`opportunity`	Listings, sessions and occurrences, revisions, the search projection, collections, saved items, interests, reminders	Built
Enrollment	`enrollment`	Family profile, shares, forms, eligibility, applications, waivers and signatures, seats and waitlists, enrolments, fees and payments	Built
Attendance	`attendance`	Attendance records, sync batches, check-in tokens, devices	Built
Notifications	`notify`	Push devices, preferences, the inbox, deliveries	Built
Community	`community`	Conversations, messages, broadcasts, screening holds (M3). Spaces, posts and moderation are M5	Messaging built
Learning	`learning`	Learner profiles, participation, issuers and keys, badge classes and assertions, playlists and progress, unlock rules (D65 to D67). Goals, skills, projects and exports to come	Partly built
Workforce, Landscape, Analytics, Integrations	`workforce`, `landscape`, `analytics`, `integration`	Mentors and gigs; asset mapping and surveys; suppressed rollups; external systems	Schema only

Dependencies. A module may reference the contracts of the modules beneath it in this order, and never the other way:

Module	May reference the contracts of
Platform	none
Identity	Platform
Directory	Platform, Identity
Opportunities	Platform, Identity, Directory
Notifications	Platform, Identity
Enrollment	Platform, Identity, Directory, Opportunities
Attendance	Platform, Identity, Directory, Opportunities, Enrollment
Learning	Platform, Identity, Directory, Opportunities, Enrollment, Attendance
Community	Platform, Identity, Directory, Opportunities, Learning

`ModuleBoundaryTests` in `BrightTime.Architecture.Tests` fails the build on any other reference, and on a module referencing another module's implementation rather than its contracts.

How modules talk

1. **Asking.** A module calls another's query interface (`IDirectoryQueries`, `IEnrolmentQueries`, `IAttendanceQueries`, …), implemented in the owning module and registered by the host. It returns contract records. No `DbContext` crosses a module line, and application SQL never joins across schemas. The one exception is the finder's search projection, `opportunity.listing_search`, a read model fed by messages.
2. **Reacting.** The originating module publishes a contract message in the same transaction as its change:

```
await db.PublishAsync(tenant.TenantId, new AttendanceChanged(enrollmentId), ct);
```

```
  The message lands in `platform.outbox_message`. The worker's dispatcher fans it out to one
  delivery job per handler (`IModuleMessageHandler<T>`, registered with
  `services.AddMessageHandler<TMessage, THandler>()`), so one failing handler retries alone
(D27).
```

Messages and their handlers today:

Message	Published by	Handled by
`SiteChanged`, `OrganisationStatusChanged`, `TaxonomyTermChanged`	Directory, Platform	Opportunities: re-projects affected listings into search
`NotificationRequested`	Any module	Notifications: the inbox and push, under the person's preferences and quiet hours. Email delivery of notifications isn't built yet
`AttendanceChanged`	Attendance (every write path)	Learning: rebuilds that enrolment's participation record
`EnrolmentAccountChanged`	Enrollment (linking a member to an account)	Learning: moves participation to or from the account

3. **Asking upwards.** Sometimes a lower module needs an answer only a higher one has. The lower module states the question as an interface in its own contracts, and the higher module implements it. Eligibility (Enrollment) asks what a young person has achieved through `IAchievementQueries`; Learning answers it (D67). Enrollment registers a default that answers "nothing", and a host that runs Learning replaces it.

Tenancy

A tenant is a county or region. Every request belongs to exactly one:

- **By host name** (`Tenancy:Mode=Host`, the default). The request's host is looked up in `platform.tenant_domain`, the only table readable before a tenant is known. An unknown host is a 404, which is why `localhost` alone shows nothing in development: use `kettle.localhost`.
- **Fixed** (`Tenancy:Mode=Fixed` with `Tenancy:TenantId`), for a single-tenant install.

The tenant's own address (`SiteContext.BaseUrl`) is built from its primary domain, never from the request's `Host` header. Every link BrightTime puts in an email, a calendar file, a passkey relying party or a badge comes from it.

Inside the database, the tenant is a transaction-local setting (`brighttime.tenant_id`) that every module context sets at the start of each transaction. Module code therefore reads inside `db.InTransactionAsync(...)`. See data-and-security.md for the policies that use it.

The request pipeline

Api (`src/BrightTime.Api/Program.cs`), in order:

1. Exception handler and status-code pages; HSTS and HTTPS redirection outside Development (health probes are exempt).

2. `ApiTenantMiddleware`: host → tenant.
3. Rate limiting: the `public` policy (300 requests a minute per client by default) and the sign-in policy (10 a minute).
4. Authentication: bearer JWT.
5. `ActorMiddleware`: the signed-in person, their memberships and capabilities; sets `brighttime.actor_id` for the database.
6. `ApiLocaleMiddleware`: the response language (`?locale=`, then the person's choice, then `Accept-Language`, then the tenant default), reported in `Content-Language`.
7. Authorization: policies and requirement handlers.
8. `AuditFlushMiddleware`: the request's audit rows are written before the response.
9. Endpoints, under `/api/v1`.

The Web host runs the same sequence with cookie authentication, a remembered language cookie, antiforgery, and Blazor's static server rendering.

The outbox and the job queue

- **Queue.** `platform.job`, a hand-rolled Postgres queue (D2): `JobRunner` claims due jobs with `FOR UPDATE SKIP LOCKED`, holds a lease (`Jobs:LockDuration`), retries with backoff, and marks a job `dead` after its last attempt.
- **Outbox.** `OutboxDispatcher` polls `platform.outbox_message` every second and enqueues one delivery job per registered handler.
- **The worker's role.** `brighttime_jobs` has a permissive policy on the queue and outbox tables only, so it can claim across tenants there and nowhere else. Every job runs under its message's tenant.
- **Scheduled work** runs as jobs or background services in the worker: reminders, capability recomputation on birthdays and after a terms change (`TermsSweep`), Stripe events, listing projection, and audit partition upkeep (`AuditPartitionMaintenance`).
- **Health.** `/health/ready` reports the oldest undispatched message and the oldest due job, degraded past one and five minutes (`Health:*`).

The finder

- Public search reads only `opportunity.listing_search`, the projection, through the `brighttime_finder` role (`BYPASSRLS`, select on that one table, D13), entered with `SET LOCAL ROLE`. The query is `db/queries/search-listings.sql`.
- Text search uses English and Spanish stemming, a trigram arm for the other four languages, and typo tolerance by strict word similarity (D6).
- The map is a server-drawn SVG of the tenant's neighbourhoods: no tiles, no coordinates (D21).

- Paging is by cursor, and every page of one search is computed at the same instant (`SearchQuery.AsOf`), so relevance can't drift between pages.
- Measured plans for seven scenarios at 50,000 listings are committed under `db/explain/`, and CI fails on a sequential scan of `listing_search`.

The app

`BrightTime.Mobile` is a .NET MAUI Blazor Hybrid host for Android and iOS. Its screens and API client are in `BrightTime.Ui.Shared`, which runs and is tested without a device.

- It talks to the Api host only, with 10-minute access tokens and rotating refresh tokens kept in the platform's secure storage.
- Passkeys use Credential Manager on Android and the system API on iOS. External sign-in runs in the system browser and hands the app a one-time code bound to its PKCE challenge (D41).
- The offline roster (D52) seals marks with AES-GCM under a key in the keychain, syncs them once through `/org/{orgId}/attendance/batch`, locks after 72 hours without contact, and wipes itself when the server says the device is revoked.
- Debug builds talk to `http://10.0.2.2:8000/api/v1/` as `kettle.localhost`, which is the compose stack seen from the Android emulator.

Localisation

- Six locales: `en`, `es`, `zh-Hans`, `bn`, `ht`, `ar`. Arabic is right to left throughout.
- Catalogues are JSON with ICU plurals (D20) in `src/BrightTime.Localisation/Strings/`. Every key must exist in all six (`CatalogueTests`). Text awaiting review is listed by key prefix in each catalogue's `@@drafts`.
- A tenant overrides any string per locale (**Wording** in the organiser console), stored in `platform.resource_override` and loaded per request.
- The API returns error codes rather than prose. Clients show `error.<code>` from the catalogue in the person's language (see api.md).
- Listing content is translated per listing (`listing_translation`). Machine translations are marked as such on the page.

CHAPTER 3

Data and security

The threat model behind all of this is plan/07-threat-model.md (threats T1 to T22). Before changing schema or authorisation, find the threat the code is answering there.

The schema

The DDL in `db/schema/` is the source of truth (Q12): SQL first, with EF Core mapped to it, and `SchemaDriftTests` failing on any difference in either direction. There are no EF migrations. `db/apply-schema.sh` applies the files in order, in one database.

File	Schema	Holds
`00-extensions-roles.sql`		Extensions (PostGIS, citext, pg_trgm, unaccent and others) and the cluster roles
`10-platform.sql`	`platform`	Tenants, domains, locales, flags, wording overrides, geography, taxonomy, media, audit, outbox, jobs, retention, subject requests, API clients, key tables
`20-identity.sql`	`identity`	Accounts, credentials, refresh tokens, roles, guardian links, consent, youth capabilities, access grants, terms
`30-directory.sql`	`directory`	Organisations, sites, hours, memberships, invitations, clearances, vetting
`40-opportunity.sql`	`opportunity`	Listings, translations, sessions, occurrences, revisions, the search projection, collections, saved items, reminders
`50-enrollment.sql`	`enrollment`	Family profile, shares, forms, eligibility, applications, waivers, seats, enrolments, fees and payments
`55-attendance.sql`	`attendance`	Records, sync batches, check-in tokens, devices
`60-learning.sql`	`learning`	Learner profiles, participation, issuers and keys, badges, playlists, projects
`70-community.sql`	`community`	Conversations, messages, broadcasts, spaces, posts, moderation
`80-workforce.sql`, `90-landscape.sql`, `95-analytics.sql`		Later milestones
`96-notify.sql`	`notify`	Push devices, preferences, notifications, deliveries
`97-integration.sql`	`integration`	Imports and external systems
`99-rls-and-grants.sql`		**The whole isolation surface**: every policy and grant, and the security-definer functions

187 tables in all. Conventions (plan/02):

- UUIDv7 keys, generated by the application.
- `tenant_id` on every tenant-owned table.
- `timestamptz` throughout, always UTC.
- `citext` for case-insensitive names and slugs.
- Checks and partial unique indexes for the invariants.
- The collation is `C.UTF-8`, because citext comparison and index order depend on it.

Database roles

Role	Used by	Row-level security
`brighttime_owner`	Owns every object	Bound by `FORCE ROW LEVEL SECURITY`
`brighttime_migrator`	Applying DDL; owns the security-definer functions	`BYPASSRLS`. The only principal that crosses tenants
`brighttime_app`	Web and Api requests	Tenant policy plus the actor guard
`brighttime_auth`	The sign-in path only	Tenant policy, exempt from the actor guard, granted on the Identity stores and nothing else
`brighttime_jobs`	The worker	Tenant policy, no actor guard. Permissive policies let it claim across tenants on `platform.job` and `platform.outbox_message` only
`brighttime_readonly`	Civic and research reads	Suppressed views only
`brighttime_finder`	The public finder, entered with `SET LOCAL ROLE` (D13)	`BYPASSRLS`, with `SELECT` on `opportunity.listing_search` and nothing else; a migration check fails if it ever gains more

Hosts log in as `bt_*_login` roles that are members of these (`deploy/postgres/init-brighttime.sh` creates them for compose).

Row-level security

Three layers, all in `99-rls-and-grants.sql`:

1. **Tenant isolation**, permissive, on every table with a `tenant_id`: `tenant_id = platform.current_tenant_id()`.

2. **Through the parent**, permissive, on child tables without a `tenant_id`, such as `issuer_key` through `issuer_profile`: an `EXISTS` against the already-protected parent.
3. **The actor guard**, *restrictive*, on youth-record tables: `platform.has_actor()`. A request with no signed-in person can't read or write a row in them, whatever reaches the query builder. Restrictive matters: a second permissive policy would be OR-ed and weaken isolation.

The tenant and actor are transaction-local settings (`set_config(..., true)`), set at the start of each module transaction, so a pooled connection never carries them forward.

The actor guard covers, among others: `identity.app_user`, guardian links, consents, capabilities and access grants; `enrollment.family_profile`, `family_member`, `family_contact`, `profile_share`, `application`, `enrollment`, `waiver_signature`, `payment_intent`; all of `attendance`'s records, batches and tokens; `learning.participation_record`; community conversations and messages; and notifications and push devices. The full list is in the file.

Security-definer functions. Where an anonymous request must see a narrow slice of a guarded table, a function owned by `brighttime_migrator` returns exactly that slice, checking the tenant and the conditions itself:

- `platform.queue_health()`: counts and ages for the readiness probe, never a row.
- `learning.public_participation(tenant, slug)`: the unhidden records of a portfolio its owner chose to show (D65).
- `learning.public_skills(tenant, slug)`: that portfolio's skills: the term, the level, and whether an organisation endorsed it, never which (D69).
- The audit partition functions, executable by the jobs role only.

`learning.badge_assertion` sits outside the actor guard on purpose, so public verification can read it in the tenant without an actor. The public routes return only what the credential itself carries (D66).

Rules the database enforces

These are constraints and triggers, not application code, so no bug above the database can break them:

- **No adult alone with a minor in a conversation** (T1). `enforce_safe_conversation()` refuses at commit every route to such a pair. `set_participant_minor()` decides who is a minor from the account, never from the caller (D53).
- **Append-only evidence.**
  - `audit_event` partitions are sealed as they are created.
  - `opportunity.listing_revision` refuses updates.
  - `enrollment.waiver_signature` is append-only, and its references restrict deletes.
  - Attendance corrections supersede the old record rather than replace it.

- **The last seat goes to exactly one applicant.** Seats are taken under a `FOR UPDATE` lock on the seat pool (D48).
- **No youth home address or coordinates** anywhere in the schema.
- **Clearances are status and expiry only**, never a background-check report.
- **No cardholder data.** Payment rows hold the gateway's ids and statuses.
- **Keys follow the module order.** A cross-schema foreign key goes to one of plan/01's nine shared reference tables, or into a module the owning module may reference: `attendance.attendance_record` → `enrollment.enrollment`, say, but never the other way. `CrossSchemaForeignKeyTests` reads the catalogue and holds it (D70).
- **Public work is approved work.** A project is `public` or `link_only` only when it's approved and published (`ck_project_public_needs_approval`); changing it takes it back to a private draft (D69).
- **Badges.** A badge is awarded once per recipient while unrevoked. A revocation always has a reason. A credential has a proof. Issuer keys are `Ed25519`, `ES256` or `RS256`, with at most one active key per issuer.

Encryption and keys

Field encryption. Sensitive fields are AES-GCM encrypted by the application before they reach the database (`AesGcmFieldCipher`):

- family members' names and dates of birth;
- support-need detail;
- family contacts' names, phones and emails;
- sensitive application answers (sealed);
- account phone numbers;
- push device tokens.

Lookups on an encrypted value use a keyed **blind index** (for example `identity.app_user.phone_blind_index`, keyed by `platform.blind_index_key`).

Tenant data keys are derived per tenant with HKDF-SHA256 from a master key in the vault (`DerivedTenantDataKeyProvider`): the tenant id is the salt and the version is in the info. `Security:DataKeys` lists the master key versions held, each with a vault reference. The highest version encrypts, and every listed version still decrypts.

The vault. `IKeyVault` resolves a reference to key material, so only references are ever stored (`issuer_key.private_key_ref`, `blind_index_key.key_ref`). `ConfiguredKeyVault` reads it from configuration, for development and single-server installs. A managed vault (Azure Key Vault, AWS KMS) can replace it behind the same interface.

Kind	Configuration	Used for
Symmetric, base64, 32 bytes or more	`Security:Keys:<ref>`	Tenant data master keys, the data-protection key ring (`data-protection`), blind indexes
RSA private key, PEM, 2048 bits or more	`Security:SigningKeys:<ref>` or `Security:SigningKeys:default`	Open Badges credentials and CLRs, signed RS256 (D66, D68); each organisation's reference is `badge-issuer-{orgId:N}`, the site's `clr-issuer-{tenantId:N}`

In Development a missing key is made up for the life of the process. Anywhere else a missing key stops the operation. See operations.md for rotation.

The data-protection key ring (cookies, antiforgery, one-time tokens) is shared by Web and Api. It lives in `platform.data_protection_key`, encrypted under `Security:Keys:data-protection`.

Badge signing. Each organisation issues under its own `did:web` DID. Its public key is published in the DID document, and its key id is the RFC 7638 thumbprint. When the vault's key changes, the old key is retired, not deleted, so older credentials still verify.

Signing in

	Web	Api (the app)
Session	Cookie	10-minute JWT (HMAC-SHA256) plus a rotating refresh token
Password	Argon2id hashes; optional breached-password check (`Auth:CheckBreachedPasswords`, k-anonymity)	Same store
Passkeys	WebAuthn	The same store and relying party (`Passkeys:AppOrigins` lists the app's origins)
External	OIDC providers under `Auth:External`	The web flow in the system browser, then a one-time PKCE-bound code (`POST /auth/external/token`)
Development	`/dev/sign-in`, only in Development with `Auth:DevelopmentSignIn=true` (D14)	

- **Refresh tokens** rotate on every use and are chained in families (D28). Presenting a spent token revokes the whole family and is audited. An adult's session ends after 30 days idle, a young person's after 30 minutes (T14). No session outlives 90 days.
- **Access tokens** are signed with the first key in `Auth:Tokens:SigningKeys`, and every listed key verifies. To rotate, add a new key first, then remove the old one after the access lifetime.

- **Account changes.** A change to an account (a password, a passkey, "sign out everywhere") changes its security stamp, which every request checks. Old tokens and cookies stop working at their next request.
- **Step-up.** Changing how an account signs in needs a sign-in within the last `Auth:StepUpMinutes` (10 by default).
- **Rate limits.** Sign-in, forgot-password and reset are limited per client address (`Auth:SignInAttemptsPerMinute`, 10 by default). The forgot-password answer is the same whether or not the address exists.
- **Registration** asks for a birth month only. Under 13 is refused; 13 to 17 makes a youth account. A child under 13 exists only as a profile on a guardian's account until a guardian consents (email plus, or an organiser-attested signed form, D31) and, at 13, hands it over (D35).

Permissions

Three separate questions, all of which must be yes (plan/04):

1. **Tenant.** Is the request in the tenant that owns the data? RLS answers this, and the application checks it too.
2. **Role.** Does one of the person's roles grant the permission? Roles are tenant-scoped rows, and the permission vocabulary is in code (`Permissions`, with defaults in `BrightTimeRoles`). `platform_admin` holds no youth-record permission at all.
3. **Scope.** Does this person's scope cover this object?
   - **Organisation scope** (`OrganisationScopeRequirement(permission, minimumRole)`): a live membership in the organisation with at least that role. The order is owner > admin > staff > mentor > viewer.
   - **A family's share**: an organisation sees a family's members and fields only through a live `profile_share` naming them (D44, D48).
   - **Guardianship**: a verified, unrevoked guardian link.
   - **Capabilities**: a youth account's `youth_capability` (enrol, be messaged, post, upload, showcase), recomputed on birthdays, consent changes and terms changes.

Refusals carry a reason code (`badges.forbidden`, `family.link_forbidden`, …) rather than an empty result: an honest caller should be told why.

Audit

- **One row per operation** in `platform.audit_event` (D11, D19): who, acting for which organisation, what action, the outcome (allow, deny, error), the subject kind and ids, a reason where required, and detail.

- It is partitioned by month. The worker creates partitions months ahead (`AuditPartitionMaintenance`), and each is sealed as it is created (`platform.seal_audit_partition`), so nothing can update or delete a row.
- Client addresses and user agents are kept only as keyed hashes (`Audit:ClientHashKey`, required outside Development).
- **Youth-record access** is declared, not remembered:
  - an endpoint or page that touches youth records carries `[YouthRecord(action)]`;
  - a DTO carrying youth data carries `[YouthData]`;
  - a guard at start-up refuses to run a host with an endpoint that can return youth data without the attribute;
  - the module writes the access with `AuditTrail.RecordInTransactionAsync` in the same transaction as the read, so there is no read without its record.
- Refusals are audited too (`Outcome = Deny`).
- The organiser console's **Audit log** reads it, filtered by action prefix, person, subject, outcome and date.

Youth safety and privacy, end to end

Concern	Control	Where
Grooming through messaging (T1)	No adult alone with a minor; an admin observes or it's refused; contact-exchange attempts to a minor held; pushes say only who wrote	Database trigger, Community module
Where children are (T4, T6)	No youth addresses; private sites show only a neighbourhood; the finder reads a projection without people in it; check-in codes name nobody	Schema, finder role, Attendance
Over-reach by providers (T8)	Names and fields only through a live family share; every read audited	Enrollment, audit
Unsafe adult gains access (T9)	Verified guardian links, revocable; consent per scope with the exact text hashed	Identity
Insider abuse (T10)	Platform admin has no youth permissions; audit is append-only and sealed	Roles, audit
Offline devices (T12)	Sealed marks, a keychain key, a 72-hour lock, remote wipe	App, Attendance
Shared devices (T14)	30-minute youth sessions; refresh reuse revokes the family	Identity, Api
Small-cell re-identification (T17)	Reports hide counts under the tenant's threshold, with complementary suppression, and say what they hid	Reporting
Payment scope (T20)	Card entry on Stripe Checkout's hosted page; webhooks believed only by signature	Enrollment
Account takeover (T21)	Passkeys first, Argon2id, breached-password check, step-up, rate limits	Identity, Web, Api
Portfolio and badges (T5)	Private by default; display names without the family name; recipients only as salted hashes	Learning

Known gaps

- **Re-encryption on data key rotation** is described in the key provider's design but not built: adding a version keeps old data readable, and nothing rewrites it yet.
- **Deletion versus audit** (Q5) and **retention periods** (Q4) are open. Nothing deletes subject data yet.
- **Deleting organisations, listings or community reports** would fail against append-only tables (D46). That needs a decision before anything deletes them.

CHAPTER 4

API reference

The API host serves everything under `/api/v1` on each tenant's own host name, for example `https://kettle.example.org/api/v1/`. It is what the app uses. 196 routes are listed below, grouped by area. In Development the host also serves the full OpenAPI document at `/api/v1/openapi.json`.

Conventions

- **Tenant.** The host name selects the tenant. A request to an unknown host is a 404.
- **Auth.** `Authorization: Bearer <access token>` on every route not marked *public*. Public routes are anonymous and rate-limited per client address (`Api:PublicRequestsPerMinute`, 300 by default); a refused request is a 429 with `Retry-After: 60`.
- **Language.** `?locale=`, then the signed-in person's own language, then `Accept-Language`, then the tenant's default. The response states the one used in `Content-Language`.
- **JSON.** camelCase. Instants are ISO 8601 with an offset, always UTC. Dates are `yyyy-MM-dd`. Ids are UUIDs.
- **Paging.** Lists that page return `{ "items": [...], "nextCursor": "..." }`. Pass `cursor` back for the next page. A cursor fixes the instant the search was computed at, so pages don't drift.
- **Idempotency.** `POST /applications` and `POST /payments/intents` require an `Idempotency-Key` header. A retry with the same key returns the first result rather than creating another. Elsewhere, retries are made safe by the operation itself: attendance batches are deduplicated per mark, a badge is awarded once, a decision is a state change.
- **Versioning.** Additive. New fields and routes may appear in `v1`; unknown query parameters are ignored.
- **Youth records.** Routes that return or change a young person's data are audited on every call (see data-and-security.md).

Errors

Errors are RFC 9457 problem documents with a stable `type` per reason:

```
{
  "type": "urn:brighttime:problem:badges.already_awarded",
  "title": "This young person already holds this badge.",
  "status": 409,
  "traceId": "00-…"
}
```

Status	When
400	Validation. `urn:brighttime:problem:validation` carries `errors` keyed by field; a module's refusal carries its own code (`badges.criteria`)
401	No valid token
403	Signed in but not allowed. The code says why (`badges.forbidden` , `portfolio.showcase` , `family.link_forbidden`)
404	Not found, or not yours to see
409	A conflict with the current state (`badges.not_published` , `seats.full`)
412	A stale version on a concurrent edit
429	Rate limited

The `title` is English. A client shows the person the catalogue string `error.<code>` (for example `error.badges.already_awarded`) in their language, falling back to the title. That's what the app's `Words.Problem` does. Values a message needs travel as extensions on the problem.

Authentication

Route		
`POST /auth/register`	public	`{ email, birthMonth }` . Under 13 is refused; 13 to 17 makes a youth account. Emails a link to choose a password
`POST /auth/token`	public	`{ email, password, deviceId }` → `{ accessToken, refreshToken, … }` . Rate-limited
`POST /auth/token/refresh`	public	`{ refreshToken }` . Rotates the refresh token; a reused token revokes its family
`POST /auth/logout`		Ends this session
`POST /auth/passkeys/assert/options` , `POST /auth/passkeys/assert`	public	Sign in with a passkey
`POST /auth/passkeys/options` , `POST /auth/passkeys`		Add a passkey
`GET /auth/external/providers`	public	The external providers this tenant offers
`POST /auth/external/token`	public	Redeem the one-time code from the web's external flow, with the PKCE verifier
`POST /auth/password/forgot` , `POST /auth/password/reset`	public	The reset email, and setting a new password with its token. The forgot answer is the same for every address
`POST /auth/claim`	public	A young person takes over the profile a guardian made, with the claim token

Public reads

All anonymous and rate-limited.

Route	
`GET /public/tenant`	Branding, locales and feature flags
`GET /public/listings`	Search. Parameters: `q`, `kind` (repeatable), `category` (repeatable, any of), `accommodation` (repeatable, all of), `age`, `grade`, `from`, `to`, `free`, `maxCost` (whole dollars), `mealsProvided`, `transportProvided`, `dropIn`, `nearNeighbourhood` (a code), `orgId`, `sort`, `cursor`, `limit`. Malformed values are a 400; an unknown kind or a stale cursor is dropped and reported in `notices`
`GET /public/listings/{slug}`	One listing
`GET /public/listings/{slug}/eligibility`	Its eligibility rules, as families see them
`GET /public/listings/{slug}/form`	Its registration form, in the request's language
`GET /public/collections`, `GET /public/collections/{slug}`	Curated collections
`GET /public/neighbourhoods`, `GET /public/neighbourhoods/{code}`	Neighbourhoods, and one with its boundary and what's on
`GET /public/organisations/{slug}`	An organisation's public page
`GET /public/taxonomy/{vocabulary}`	`category`, `interest`, `accommodation` or `skill`, named in the request's language
`GET /public/portfolios/{slug}`	A portfolio its owner chose to show (D65). `noindex` when link-only
`GET /public/issuers/{orgId}/did.json`	An organisation's DID document (`application/did+json`), every non-compromised key (D66). With the tenant's own id, the site's, which signs CLRs (D68)
`GET /public/issuers/{orgId}/ob2`	The OB 2.0 Issuer
`GET /public/issuers/{orgId}/revocations`	The `1EdTechRevocationList` of its revoked credentials (`no-store`)
`GET /public/badges/classes/{id}`	A published badge as an OB 3.0 Achievement

Route	
`GET /public/badges/classes/{id}/ob2`	As an OB 2.0 BadgeClass
`GET /public/badges/classes/{id}/image.svg`	Its generated image
`GET /public/badges/{slug}`	A credential's standing (`valid` , `expired` , `revoked`), with the credential and its JWT. `no-store` , `noindex`
`GET /public/badges/{slug}/ob2`	The OB 2.0 hosted assertion, or `410 Gone` with `revoked: true` and the reason
`GET /public/playlists` , `GET /public/playlists/{slug}`	Published playlists: steps, estimated hours, the badge at the end, and any badge needed to start (D67)
`GET /terms`	The current terms of use, with their hash

Your account

Route	
`GET /me`	The signed-in person
`GET /me/capabilities`	What the account may do now (enrol, be messaged, post, upload, showcase), so the app gates on the server's answer
`GET /me/terms` , `POST /me/terms`	Whether the person has agreed to the current terms, and agreeing (by hash)
`DELETE /me/sessions`	Sign out everywhere
`GET /me/passkeys` , `DELETE /me/passkeys/{id}`	The account's passkeys
`GET /me/feed`	The personal feed, paged, each item with why it was shown
`GET` , `PUT /me/interests`	Interests, with weights
`GET` , `PUT /me/search-preferences`	The anchor neighbourhood and distance
`GET /me/saved-items` , `PUT` , `DELETE /me/saved-items/{listingId}`	Saved listings, with an optional note
`GET` , `POST /me/reminders` , `DELETE /me/reminders/{id}`	Reminders before a session or a deadline, booked as jobs
`GET /me/notifications` , `POST /me/notifications/{id}/read`	The inbox
`GET` , `PUT /me/notification-preferences`	Channels per kind, and quiet hours

Children and consent

Route	
`GET`, `POST /children`, `GET /children/{id}`	A guardian's child profiles: a given name and a birth month only
`GET /consent/disclosures`	The exact text of each consent scope, with its hash
`POST /children/{id}/consents`, `DELETE /children/{id}/consents/{scope}`	Give consent to a scope (by the disclosure's hash), or withdraw it
`POST /children/{id}/claim`	Hand a child of 13 or over their own sign-in, by email
`POST /guardian-links/{id}/verify`	Confirm the guardianship with the consent email's token
`POST /guardian-links/{id}/consent-email`	Send the consent email again
`POST /guardian-links/{id}/revoke`	Stop managing the profile; ends every consent

Family profile and shares

The owner alone reads or changes a family profile.

Route	
`GET`, `PUT /family-profile`	The household
`POST /family-profile/members`, `PUT`, `DELETE /family-profile/members/{id}`	Members; names and dates of birth are encrypted
`PUT /family-profile/members/{id}/support-needs`	A member's support needs
`PUT /family-profile/members/{id}/account`	Link a youth or child member to the owner's own account or a verified-guardian child's (`{ userId }`, `null` unlinks; D65)
`POST /family-profile/contacts`, `PUT`, `DELETE /family-profile/contacts/{id}`	Emergency and other contacts
`GET`, `POST /family-profile/shares`, `DELETE /family-profile/shares/{id}`	What the family shares with an organisation: members, fields, optionally one listing, until a date. Deleting withdraws it at once

Applications, waivers and payments

Route	
`GET /applications`, `POST /applications`	The family's applications; start a draft (`Idempotency-Key` required)
`GET`, `PATCH /applications/{id}`	One application; answer its form
`GET /applications/{id}/form`	The questions it answers: the form version the draft began on, in the family's language (D72)
`POST /applications/{id}/submit`	Submit once: checks completeness, hard eligibility rules and required waivers, and shares the applicant as chosen
`POST /applications/{id}/withdraw`	Withdraw
`POST /applications/{id}/offer/accept`, `/offer/decline`	Answer a place offered from the waitlist
`GET /applications/{id}/waivers`, `POST /applications/{id}/waivers/{waiverId}/sign`	The waivers it needs, and signing one (by the text's hash)
`GET /applications/{id}/fees`	The fees and price levels that apply
`GET /applications/{id}/payments`	Its payments, newest first (D73)
`POST /payments/intents`, `GET /payments/intents/{id}`	Pay a fee at a price level (`Idempotency-Key` required). With Stripe, returns `hostedPaymentUrl`, Stripe Checkout's page for the phone's browser (D74); otherwise the payment waits to be marked received. A one-time or seasonal fee already paid or waived is `payments.already_paid` (D73)

Attendance

Route	
`GET /me/attendance`	A family's own attendance
`POST /me/attendance/badges`	A two-minute, single-use check-in code for an enrolment, for staff to scan
`POST /attendance/checkin`	Check in by scanning a kiosk's one-minute code
`POST /attendance/devices/contact`	A device's check-in with the server, without signing in: says whether it's revoked and must wipe

Messages and devices

Route	
`GET /conversations`, `GET /conversations/{id}`	The person's conversations
`POST /conversations/{id}/messages`	Reply. A message from an adult to a minor may be held for a moderator
`POST /devices`, `DELETE /devices/{id}`	Register a phone for push, and forget it

Learning: portfolio, badges and playlists

Route	
`GET`, `PUT /me/portfolio`	The young person's portfolio and its settings (display name, headline, bio, visibility, link name)
`POST /me/portfolio/participation/{id}/hide`, `/show`	Hide a record from the total and any shown page, or show it again
`POST /me/portfolio/export`	`{ format }`: `pdf` (a printable portfolio in the person's language), `json_ob3` (their badges' OB 3.0 credentials and JWTs) or `clr2` (a CLR 2.0 VC-JWT signed by the site, `application/jwt`; needs at least one badge). Returned at once, never stored (D68)
`GET /me/badges`	Badges held, revoked ones marked, each with its credential JWT
`POST /playlists/{id}/enroll`	Start a published playlist: needs the account's sign-up capability and any badge the playlist asks for. Starting one already started returns it
`GET /me/playlists`	Playlists started, each step with its state (`not_started`, `submitted`, `completed`, `needs_rework`), whether it's open yet, and any reviewer's note
`PUT /me/playlists/{id}/steps/{stepId}`	Mark a step done (`{ reflection }` for a reflection step): completed, or submitted when it needs review. Completing the last required step completes the playlist and awards its badge

Learning: goals, skills and projects

Route	
`GET`, `POST /me/goals`; `PUT /me/goals/{id}`	Private goals: a title, detail, an optional interest or skill term, a target date, and `active`, `achieved`, `paused` or `abandoned` (D69)
`GET /me/skills`	Skills, each with its level (1 to 4), and whether and by which organisation it was endorsed
`POST /me/skills`	`{ termId, level }`: list a skill from the `skill` vocabulary as one's own, or change its level
`DELETE /me/skills/{id}`	Remove a skill, endorsed or not
`GET`, `POST /me/projects`; `PUT`, `DELETE /me/projects/{id}`	Projects: a title, a description, and the listing it came from (one the person took part in). Changing an approved or published project makes it a private draft again
`POST /me/projects/{id}/submit`	Ask the listing's organisation to approve it
`PUT /me/projects/{id}/visibility`	`{ visibility }`: `public` or `link_only` once approved (needs the showcase capability), or `private`

Organisation routes

All under `/org/{orgId}`. Each needs a live membership in that organisation with the permission and minimum role shown. *Admin* means owner or admin.

Route	Who	
`GET /family-shares`, `GET /family-shares/{id}`	Staff	Families' live shares with the organisation
`GET`, `POST /forms`; `GET`, `PUT /forms/{id}`; `POST /forms/{id}/publish`, `/revise`, `/retire`	Admin	Registration forms
`PUT`, `DELETE /listings/{listingId}/form`	Admin	Attach or detach a published form
`GET`, `POST /waivers`; `GET`, `PUT /waivers/{id}`; `POST /waivers/{id}/publish`, `/revise`, `/retire`	Admin	Waivers
`PUT`, `DELETE /listings/{listingId}/waivers/{waiverId}`	Admin	Ask for a waiver on a listing, required or optional

Route	Who	
`GET` , `PUT /listings/{listingId}/eligibility` ; `POST …/eligibility/preview`	Admin	Rules (`age_at_start` , `grade_range` , `neighbourhood_in` , `badge_required` , `playlist_completed`), and a dry run against a made-up applicant, who may hold `badgeClassIds` and `playlistIds`
`GET /listings/{listingId}/applications` ; `GET /applications/{id}`	Staff	The queue, and one application as the family's share allows
`POST /applications/{id}/decide` , `POST /applications/bulk-decide`	Staff	Decide singly or in bulk; the bulk answer lists any left and why
`GET /listings/{listingId}/seats` ; `POST /listings/{listingId}/waitlist/offer`	Staff	Places taken and waiting; offer freed places to the front of the waitlist
`PUT /listings/{listingId}/seats`	Admin	Set places and waitlist places
`GET` , `PUT /listings/{listingId}/fees` ; `GET /listings/{listingId}/payments`	Admin	Fees and price levels; payments
`POST /payments/{id}/received` , `POST /payments/{id}/waive`	Admin	Mark received; waive with a reason
`GET /attendance/meetings` ; `GET /listings/{listingId}/roster`	Staff	Meetings, and a meeting's roster
`GET` , `POST /attendance` ; `PATCH /attendance/{id}`	Staff	Records; record a mark; correct one with a reason
`POST /attendance/batch`	Staff, registered device	An offline batch. Each mark comes back `accepted` , `duplicate` , `conflict` or `rejected`
`POST /attendance/scan`	Staff	Check in by scanning a family's code
`POST /checkin-tokens`	Staff	A kiosk's one-minute code
`GET /kiosk-devices`	Admin	Phones and kiosks registered to sync
`POST /kiosk-devices` ; `POST /kiosk-devices/{id}/revoke`	Staff	Register a device; revoke one, which asks it to wipe on its next contact
`POST /conversations`	Staff	Start a conversation with a family (an admin joins when a minor is in it)
`POST /broadcasts`	Staff	An announcement to an activity's families
`GET /reports/catalogue` ; `POST /reports/runs`	Staff	Reports, made on request as PDF, Excel or CSV and never stored
`GET` , `POST /badge-classes` ; `PUT /badge-classes/{id}` (a draft only); `POST …/publish` , `/retire`	Admin to define; staff to list	Badges (D66)

Route	Who	
`GET` , `POST /badge-classes/{id}/awards`	Staff	Awards, named as the family's share allows; award one (`{ enrollmentId, narrative }`)
`POST /badge-assertions/{id}/revoke`	Admin	Revoke with a reason
`GET /projects` ; `POST /projects/{id}/review`	Staff (`projects.review`)	Projects about the organisation's activities waiting for approval, named by portfolio display name only; approve, or send back (`{ approve, note }`)
`POST /skills/endorse`	Staff (`skills.endorse`)	`{ enrollmentId, termId, level }` : endorse a skill for a young person enrolled with the organisation
`GET` , `POST /playlists` ; `GET` , `PUT /playlists/{id}` (a draft); `PUT /playlists/{id}/steps` (a draft); `POST …/publish` , `/archive`	Staff (`playlists.author`)	Playlists (D67)
`GET /playlists/{id}/submissions` ; `POST /playlists/{id}/steps/{stepId}/review`	Staff (`playlists.review_steps`)	Steps waiting for review, named by portfolio display name only; approve, or send back with a note (`{ enrollmentId, approve, note }`)

Organisations' listings, sites, staff and profile are managed in the web console, not the API.

Organiser and webhook routes

Route	
`POST /organiser/guardian-links/{id}/attest`	A network organiser confirms a signed consent form (`consent.attest_signed_form`)
`POST /webhooks/payments/{provider}`	The payment provider's webhook (`stripe`). Believed only by its signature; applied by the worker

CHAPTER 5

Configuration

Each host reads standard .NET configuration: `appsettings.json`, then `appsettings.{Environment}.json`, then environment variables, where a double underscore stands for a colon (`Security__SigningKeys__default` is `Security:SigningKeys:default`). Secrets belong in environment variables or a secret store, never in a committed file.

Development is different on purpose. With `ASPNETCORE_ENVIRONMENT` (or `DOTNET_ENVIRONMENT` for the worker) set to `Development`:

- missing keys are made up for the life of the process;
- the OpenAPI document is served;
- cookies may travel over plain HTTP;
- `/dev/sign-in` can be switched on.

Anywhere else, every missing key below marked **required** stops the host at start-up, or stops the operation that needs it.

Columns: **W** Web, **A** Api, **J** Jobs.

Connections

Setting	W	A	J	
`Finder:ConnectionString`	•			**Required** when `Finder:Mode=Sql`. A login that is a member of `brighttime_app`; the web uses it for the finder (as `brighttime_finder`) and for every module
`Auth:ConnectionString`	•			**Required.** A login in `brighttime_auth`, for sign-in
`ConnectionStrings:App`		•		**Required.** A login in `brighttime_app`
`ConnectionStrings:Auth`		•		**Required.** A login in `brighttime_auth`
`ConnectionStrings:Jobs`			•	**Required.** A login in `brighttime_jobs`

With Npgsql in containers, add `GSS Encryption Mode=Disable`: the .NET images have no Kerberos library to try it with.

Tenancy and addresses

Setting	W	A	J	Default	
`Tenancy:Mode`	●	●		`Host`	`Host` (by host name) or `Fixed`
`Tenancy:TenantId`	●	●			The tenant when `Mode=Fixed`
`Tenancy:PublicScheme`	●	●		`https`	The scheme of tenants' own addresses. Outside Development anything but `https` stops start-up
`Tenancy:PublicPort`	●	●			A port for tenants' addresses, development only (`8000` for compose)
`Auth:PublicBaseUrl`	●	●			A fixed public address, overriding the tenant's domain, for a single-site install
`Network:TrustedProxies`	●	●			Proxy addresses whose `X-Forwarded-For` and `-Proto` are believed. List the reverse proxy here in production
`Network:TrustedNetworks`	●	●			CIDR networks believed the same way, for proxies without a fixed address (containers)
`Network:ForwardLimit`	●	●		`1`	How many forwarded hops to read

Keys

Setting	W	A	J	
`Security:Keys:<ref>`	●	●	●	Base64 symmetric keys, 32 bytes or more, by vault reference
`Security:Keys:data-protection`	●	●		**Required.** Encrypts the shared data-protection key ring. Must be the same in both hosts. `DataProtection:KeyRef` names a different reference
`Security:DataKeys:<n>:Version` , `:KeyRef`	●	●	●	**Required.** The master key versions tenant data keys are derived from, each naming a `Security:Keys` reference. The highest version encrypts. Must match across all three hosts
`Security:SigningKeys:<ref>` , `Security:SigningKeys:default`		●		**Required to award badges.** RSA private keys as PEM, 2048 bits or more. `default` serves every reference without its own; per organisation, the reference is `badge-issuer-{orgId:N}` (D66); the site's, which signs CLRs, is `clr-issuer-{tenantId:N}` (D68)
`Auth:Tokens:SigningKeys:<n>:Id` , `:Key`		●		**Required.** Access-token keys: an id and a base64 key of 32 bytes or more. The first signs; all verify
`Audit:ClientHashKey`	●	●		**Required.** Base64, 32 bytes or more. Client addresses and user agents are kept only as hashes under it

Generate a symmetric key with `openssl rand -base64 32` , and an RSA key with `openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:3072` .

Sign-in and sessions

Setting	W	A	Default	
`Auth:Tokens:AccessLifetime`		●	`00:10:00`	Access token lifetime
`Auth:Tokens:RefreshIdle`		●	`30.00:00:00`	An adult's session ends after this long unused
`Auth:Tokens:YouthRefreshIdle`		●	`00:30:00`	A young person's (T14)
`Auth:Tokens:RefreshAbsolute`		●	`90.00:00:00`	No session outlives this
`Auth:SignInAttemptsPerMinute`	●	●	`10`	Sign-in, forgot and reset submissions per client address
`Auth:CheckBreachedPasswords`	●	●	`false`	The k-anonymity check against Have I Been Pwned
`Auth:StepUpMinutes`	●		`10`	How recent a sign-in a step-up action needs
`Auth:DevelopmentSignIn`	●		`false`	`/dev/sign-in`, honoured only in Development (D14)
`Auth:External:<name>:DisplayName`, `:Authority`, `:ClientId`, `:ClientSecret`	●			An external OIDC provider, such as Google. The authority is its https issuer
`Auth:AppCallback`	●		`org.brighttime.app://external`	Where the web hands the app its one-time code (D41)
`Passkeys:AppOrigins`	●	●		The app's origins (`android:apk-key-hash:…`), accepted by the shared relying party
`Api:PublicRequestsPerMinute`		●	`300`	The public routes' limit per client address

The app's links

Setting	W	
`AppLinks:AppleAppIds`	●	`TEAMID.bundle.id` entries, served in `/.well-known/apple-app-site-association`
`AppLinks:AndroidPackage`	●	The Android package name
`AppLinks:AndroidCertificateFingerprints`	●	SHA-256 signing fingerprints, served in `/.well-known/assetlinks.json`

Mail, push, payments and reports

Setting	W	A	J	Default	
`Email:Host`, `:Port`, `:EnableSsl`, `:UserName`, `:Password`	●	●	●	port `25`	SMTP
`Email:From`	●	●	●	`BrightTime <no-reply@brighttime.invalid>`	The sender
`Email:PickupDirectory`	●	●	●		Write messages to a folder instead of sending (tests and development)
`Push:Fcm:ProjectId`, `:ServiceAccountJson`			●		Push through Firebase Cloud Messaging (D40). Without them, pushes are logged, not sent
`Payments:Stripe:SecretKey`, `:WebhookSecret`		●			Both set: payments go to Stripe Checkout (D56, D74). Either missing: the manual gateway
`Payments:Stripe:ApiBase`, `:ApiVersion`, `:WebhookTolerance`		●		`https://api.stripe.com/`, `2024-06-20`, `00:05:00`	
`Reports:QuestPdfLicence`	●	●		Community	QuestPDF's licence type. Community is free only under US$1M revenue (D57)

The finder

Setting	W	Default	
`Finder:Mode`	●	`Sql`	`Sql`, or an in-memory sample for tests
`Finder:TimeZone`	●	`America/New_York`	For "starts on" dates

The worker

Setting	J	Default	
`Jobs:Queue`	•	`default`	The queue it claims from
`Jobs:BatchSize`	•	`10`	Jobs claimed at once
`Jobs:PollInterval`	•	`00:00:02`	How often it looks
`Jobs:LockDuration`	•	`00:05:00`	How long a claim is held before another worker may take it
`Jobs:WorkerId`	•	machine and process	
`Outbox:BatchSize`	•	`100`	Messages dispatched at once
`Outbox:PollInterval`	•	`00:00:01`	

Health

Setting	W	A	Default	
`Health:OutboxLagDegraded`	•	•	`00:01:00`	`/health/ready` reports degraded past this outbox lag
`Health:JobLagDegraded`	•	•	`00:05:00`	and past this job lag

A production checklist

Before a host serves real people:

1. `ASPNETCORE_ENVIRONMENT=Production`, and `Tenancy:PublicScheme=https`.
2. Connection strings for the `bt_*` logins, never the superuser.
3. `Security:Keys:data-protection` and `Security:DataKeys` the same in every host, and the referenced keys present.
4. `Auth:Tokens:SigningKeys` in the Api, `Audit:ClientHashKey` in Web and Api.
5. `Security:SigningKeys:default` (or one per organisation) in the Api, if badges are used.
6. `Network:TrustedProxies` naming the reverse proxy.
7. `Email:*` for a real relay, with a real `Email:From`.
8. `Auth:CheckBreachedPasswords=true`.
9. `Auth:DevelopmentSignIn` unset.
10. Optional: `Payments:Stripe:*`, `Push:Fcm:*`, `Auth:External:*`, `AppLinks:*`, `Passkeys:AppOrigins`, `Reports:QuestPdfLicence`.

CHAPTER 6

Operations

The local stack

Everything runs in containers with one command:

```
docker compose -f deploy/compose.yml up --build
```

Service	What it is	Reached at
`postgres`	PostGIS 18-3.6, schema applied on first boot by `deploy/postgres/init-brighttime.sh`	`127.0.0.1:5432`
`seed`	Loads Kettle County, then exits (seed.md)	
`web`, `api`, `jobs`	The three hosts, in Development	through nginx
`nginx`	Routes `/api/` to the API and the rest to Web, with the security headers	`http://kettle.localhost:8000`
`mailpit`	Catches all mail	`http://localhost:8025`
`nominatim`	Self-hosted geocoding, behind the `geocoding` profile and not started by default (Q7)	`127.0.0.1:8080`

- Browse `http://kettle.localhost:8000`. Plain `localhost` is a 404, because the tenant comes from the host name.
- Sign in to the consoles at `/dev/sign-in` as any seeded account.
- **The schema is applied on first boot only.** After any change under `db/schema/`, or a change to role permissions, rebuild the volume:

```
docker compose -f deploy/compose.yml down -v
```

Running the hosts outside containers, and the test databases, are covered in development.md and HANDOFF.md.

The database

Building one. `db/apply-schema.sh <database> [host] [port] [user]` drops and recreates the database, then applies `db/schema/*.sql` in order. It refuses any name not starting with `brighttime`. Roles are cluster-wide and created only if absent, so give each environment its own cluster.

Logins. The DDL creates only `NOLOGIN` group roles. Create a login per host and grant it the group:

```
create role bt_web_login  login password '…';
create role bt_auth_login login password '…';
create role bt_jobs_login login password '…';
grant brighttime_app, brighttime_finder to bt_web_login;
grant brighttime_auth to bt_auth_login;
grant brighttime_jobs to bt_jobs_login;
```

Seeding. `dotnet run --project src/BrightTime.Seed -- --connection "…"`, as a superuser or a member of `brighttime_migrator`. It is safe to re-run (seed.md). A real tenant needs its own tenant, domain, neighbourhood and taxonomy rows; there is no tenant-provisioning screen yet.

Upgrading a database that holds data has no tool yet. `apply-schema.sh` rebuilds from empty, and EF migrations aren't used (Q12). Before production data exists, schema changes need a forward migration path: hand-written, reviewed SQL applied as `brighttime_migrator`, with `SchemaDriftTests` proving the result.

Settings the DDL expects: collation `C.UTF-8` (citext comparison and index order depend on it), and the extensions in `00-extensions-roles.sql`. Compose also sets `shared_buffers=512MB` and `work_mem=16MB` for the finder's GIN and GiST indexes, and logs statements slower than 200 ms.

Deploying

There is no production deployment yet. What one needs, from what is built:

- **Images:** `deploy/web.Dockerfile`, `api.Dockerfile`, `jobs.Dockerfile` and `seed.Dockerfile`, built from the repository root.
- **A reverse proxy** in front of Web and Api, like `deploy/nginx/dev.conf`. It must:
  - route `/api/` (and `/hubs/`) to Api and everything else to Web;
  - pass `Host`, `X-Forwarded-For` and `X-Forwarded-Proto`;
  - add the security headers in that file (a strict CSP with no third-party origins, `nosniff`, `no-referrer`, `DENY` framing);
  - keep query strings out of the access log;
  - terminate TLS, with HSTS.

  List the proxy in `Network:TrustedProxies`.
- **One host name per tenant**, each with a row in `platform.tenant_domain`. The primary domain is the tenant's address in every link.
- **One or more workers.** Jobs are claimed with `SKIP LOCKED`, so workers can run side by side.
- **Configuration** per configuration.md.

Health and the worker

Web and Api each serve:

- `/health/live`: the process is up;
- `/health/ready`: the database answers, plus the queue's lag from `platform.queue_health()`. It is degraded when the oldest undispatched outbox message is over a minute old, or the oldest due job over five.

Health probes skip the tenant lookup and the HTTPS redirect, so a load balancer can call them by IP address.

Looking at the queue (as a superuser or `brighttime_migrator`):

```
-- what is waiting, running or dead, by type
select job_type, state, count(*), min(run_after) from platform.job group by 1, 2 order by 1, 2;

-- why the dead ones died
select id, tenant_id, job_type, attempts, last_error, created_at
from platform.job where state = 'dead' order by created_at desc limit 50;

-- outbox messages not yet dispatched
select message_type, count(*), min(occurred_at) from platform.outbox_message
where processed_at is null group by 1;
```

A job retries with backoff up to its `max_attempts` (10 by default), then is `dead`. To try a dead job again once its cause is fixed:

```
update platform.job
set state = 'ready', attempts = 0, run_after = now(), last_error = null,
    locked_by = null, locked_at = null, lock_expires_at = null
where id = '…' and state = 'dead';
```

A claim that outlives `Jobs:LockDuration` is presumed abandoned and reclaimed, so a crashed worker loses nothing.

Keys

What each key is for is in data-and-security.md.

Key	To rotate
Access-token signing (`Auth:Tokens:SigningKeys`)	Put the new key first and keep the old one second. Remove the old one after the access lifetime (10 minutes)
Tenant data master key (`Security:DataKeys`)	Add a new version with a new `Security:Keys` reference, in all three hosts at once. New writes use it; old data still decrypts. Keep every old version: nothing re-encrypts yet
Data-protection key ring (`Security:Keys:data-protection`)	The ring rotates its own keys. This key encrypts the stored ring, so changing it makes the ring unreadable: everyone is signed out and outstanding one-time links stop working. Plan it, don't just swap it
Audit client hash (`Audit:ClientHashKey`)	Changing it breaks the link between old and new hashes of the same client. Change it only deliberately
Badge signing (`Security:SigningKeys`)	Replace the PEM. The next award retires the old key (it stays in the DID document) and publishes the new one. Older credentials still verify
A compromised badge key	Mark its `learning.issuer_key` row `compromised` , which drops it from the DID document, then replace the PEM. Credentials it signed stop verifying, which is the point

Payments

Without Stripe, payments use the **manual gateway**: a family's payment waits until an admin marks it received or waives it.

To take card payments through **Stripe** (D56, D74):

1. Set `Payments:Stripe:SecretKey` and `Payments:Stripe:WebhookSecret` in the Api.
2. In Stripe, send webhooks to `https://<tenant host>/api/v1/webhooks/payments/stripe` , for `checkout.session.completed` , `.async_payment_succeeded` , `.async_payment_failed` and `.expired` , `payment_intent.succeeded` , `.processing` , `.requires_action` , `.payment_failed` and `.canceled` , and `charge.refunded` .
3. Card details are entered on Stripe Checkout's hosted page, in the phone's browser. They never reach BrightTime. Stripe returns the family to the site's `/payment/done` .
4. Try it with test keys before going live.

Webhooks are believed only by their signature, within `Payments:Stripe:WebhookTolerance` . The worker applies them, so a payment's status only moves forward. A refund made in Stripe's dashboard is reported back. Refunds started from BrightTime aren't built.

Open (D56): what to do when a waived fee is paid anyway (it is logged, not changed), and whether organisations must be paid directly (Stripe Connect).

Push notifications

Set `Push:Fcm:ProjectId` and `Push:Fcm:ServiceAccountJson` in the worker to send through Firebase Cloud Messaging (D40). Without them, pushes are written to the log. Devices that FCM reports as gone are revoked. The app's push registration also needs the Firebase project's config files in the app build. Apple delivery goes through FCM as well, which needs the Apple developer account.

Mail

Web, Api and the worker send account mail (registration, reset, consent, claim, invitations) through SMTP (`Email:*`). Locally it lands in mailpit. Notification emails aren't built yet: notifications reach the inbox and push only.

Badges

- Set `Security:SigningKeys:default` (or one key per organisation) in the Api before anyone awards a badge outside Development.
- The CLRs young people export are signed by the site itself, under the tenant's id, with the key reference `clr-issuer-<tenant>`; `Security:SigningKeys:default` serves it unless it has its own.
- Each organisation's DID resolves to `https://<tenant host>/api/v1/public/issuers/<org>/did.json`. The tenant's host name must be stable: changing it changes every issuer's DID, and older credentials stop resolving.
- **Check by hand before launch:** run a few credentials through 1EdTech's OB 3.0 validator, an exported CLR through its CLR 2.0 validator, and the OB 2.0 hosted assertions through an OB 2.0 validator. This can't run in CI.

Not yet in place

- An upgrade path for a database holding data (above).
- Backups, restores and retention. Retention periods are open question Q4; deletion versus audit is Q5.
- Object storage for media and generated files: undecided, so reports are made on request and listings have no images.
- Geocoding (Q7): new sites sit at their neighbourhood's centre.

- SMS (Q9).
- A tenant-provisioning tool.

CHAPTER 7

Development

Machine-specific setup, including the Windows scratch container on port 5434 and running the app on the Android emulator, is in HANDOFF.md.

Tools

- The .NET SDK named in `global.json` (10.0.200, rolling forward to the latest feature band).
- PostgreSQL 18 with PostGIS 3.6, or Docker (`postgis/postgis:18-3.6`).
- Docker, for the compose stack and for Testcontainers.
- On Windows, Git Bash or WSL for `db/apply-schema.sh` .
- For the app: the MAUI Android workload (`dotnet workload install maui-android`), the Android SDK and a JDK; a Mac for iOS.

Building

```
dotnet build
```

Analysers run at `latest-all` with warnings as errors, and package versions are central (`Directory.Packages.props`). The solution file is `BrightTime.slnx` .

Tests

Tests are xUnit v3 on Microsoft.Testing.Platform (`global.json` selects the runner), with Shouldly.

Project	Covers	Needs
`BrightTime.SharedKernel.Tests`	Ids, results, paging, text	nothing
`BrightTime.Localisation.Tests`	Every key in all six catalogues, ICU syntax, plurals	nothing
`BrightTime.Architecture.Tests`	The module dependency order; contracts depend only on the shared kernel; only clocks read the time	nothing
`BrightTime.Ui.Shared.Tests`	The app's screens and offline roster, without a device	nothing
`BrightTime.Infrastructure.Tests`	Key vault, field encryption, outbox, job runner, audit	a database for some
`BrightTime.Modules.*.Tests`	Each module's rules	a database for most
`BrightTime.Seed.Tests`	The seed	a database
`BrightTime.IntegrationTests`	The schema, RLS, the tenant-isolation matrix, schema drift, the database rules (the conversation safety trigger and others)	a database, or Docker for Testcontainers
`BrightTime.Web.Tests`	Pages and consoles end to end in process, the console route matrix, the youth-record guard	`BRIGHTTIME_TEST_CONNECTION`
`BrightTime.Api.Tests`	Every API area end to end in process, including D65 and D66	`BRIGHTTIME_TEST_CONNECTION`
`BrightTime.E2E.Tests`	Playwright in Chromium and WebKit: axe-core on every public route in six locales, keyboard-only use, the finder with JavaScript off	`BRIGHTTIME_TEST_CONNECTION` and installed browsers

Running them. Point the tests at a cluster of their own, never one holding real data: the fixtures create cluster-wide roles, and drop and recreate `brighttime_*` databases.

```
BRIGHTTIME_TEST_CONNECTION="Host=127.0.0.1;Port=5433;Database=brighttime;Username=postgres;Trust Server Certificate=true" dotnet test
```

- Apply the schema to that database first (`db/apply-schema.sh`), and again after any DDL change.
- **Without the variable, database tests skip silently and the run still passes.** The integration project starts its own container through Testcontainers instead; the Web, Api and E2E tests skip.
- One suite per database server at a time: the fixtures use fixed database names (`brighttime_apitest`, `brighttime_consoletest`, `brighttime_e2etest`, …).

- Run one class with `dotnet test --project tests/BrightTime.Api.Tests -- --filter-class "*BadgeApiTests"`.
- Install the E2E browsers once per machine, after a build: `pwsh artifacts/bin/BrightTime.E2E.Tests/debug/playwright.ps1 install chromium webkit`. A failure leaves a trace in `artifacts/e2e-traces/`.

The full suite is 1227 tests (at D66), all passing against PostgreSQL 18.6 and PostGIS 3.6.4.

CI

`.github/workflows/ci.yml` runs on every push to `main` and every pull request:

Job	Does
build and unit tests	Restore, a Release build, `dotnet format --verify-no-changes`, and `dotnet test`
Android app builds	Installs the MAUI Android workload and builds the app, with its format check
schema	Applies the DDL in order, checks every module table has row-level security, and checks the app role can't bypass it
finder query plans	Runs the seven finder scenarios at 50,000 listings, fails on a sequential scan of `listing_search` or estimates off by more than 10×, and keeps the plans
accessibility	The E2E gate against a PostGIS service, keeping traces of failures
web and API suites	Both suites against a PostGIS 18-3.6 service, with the `C.UTF-8` collation and 300 connections (D70)

The build job sets no `BRIGHTTIME_TEST_CONNECTION`: there the integration suite runs on Testcontainers and the database suites skip, because their own jobs run them.

Line endings. On Windows, a CRLF checkout fails `dotnet format --verify-no-changes` on every file. Check formatting as CI does, in a clone made with `core.autocrlf=false`.

Adding a feature slice

The order that has worked through D66:

1. **Decide.** Write the decision in plan/08-decisions.md as *proposed*: what, why, what deviates from the plan, and what isn't done yet.
2. **Schema.** Change `db/schema/` (and `99-rls-and-grants.sql` for any new table's policies and grants). A youth-record table goes in the actor guard's list. Rebuild your test database.
3. **Mapping.** Map the tables in the module's `DbContext` exactly as the DDL has them, including `jsonb` and `citext` column types and the indexes. `SchemaDriftTests` holds the two together.

4. **Module.** Write the service in the module, reading inside `db.InTransactionAsync`. Talk to other modules only through their contracts; publish changes with `db.PublishAsync`. Return `Result<T>` with an `Error` code for every refusal.
5. **Permissions.** Add any new permission to `Permissions` and its roles to `BrightTimeRoles`. Check organisation scope with `OrganisationScopeRequirement`.
6. **Audit.** Record youth-record access with `AuditTrail.RecordInTransactionAsync` in the same transaction. Mark endpoints and pages `[YouthRecord(action)]` and DTOs `[YouthData]`.
7. **Endpoints or pages.** API routes map to `ApiProblems.From(error)` on failure. Console pages are static SSR forms: post, then redirect with `?done=`.
8. **Words.** Every new string, including each `error.<code>`, goes into all six catalogues. Mark machine-drafted text in `@@drafts`.
9. **Tests.** At least one end-to-end test through the API or the console, as the real login (not the superuser), so RLS and the actor guard are exercised.
10. **Docs.** Update HANDOFF.md, the *Built* note in plan/05, and these documents where the slice changes them.

Conventions the build enforces

Rule	Enforced by
Modules reference only the contracts they're allowed	`ModuleBoundaryTests`
Only clocks read the system time (`IClock` , `TimeProvider`)	An architecture test
EF mapping equals the DDL, both ways	`SchemaDriftTests`
A cross-schema foreign key goes to a shared reference table, or into a module the owner may reference	`CrossSchemaForeignKeyTests` (D70)
Every module table has RLS; the app role can't bypass it	CI's schema job, `TenantIsolationMatrixTests`
Every console route refuses other tenants and other organisations	`ConsoleRouteMatrixTests`
An endpoint that can return youth data declares `[YouthRecord]`	The start-up guard, `YouthRecordGuardTests`
Every string exists in all six catalogues	`CatalogueTests`
A new public page is in `PublicRoutes`, so axe and the keyboard gate see it	`AccessibilityTests` , `KeyboardTests`
In-page links use `SamePageLink` , never a bare `#id` (`<base href="/">` sends it home)	Review, and the keyboard gate
Formatting, including `using` order (System first)	`dotnet format` in CI
No sequential scan in the finder's plans	CI's plans job

More of the traps, such as the static SSR form rule, draining queues in tests and shared test accounts, are in HANDOFF.md.